

Assessing Bicycle Lane Quality Using Smartphone Sensor Data

5ARE0 Data Analysis & Learning Methods — Assignment 1 (2026–2027)

Team 24 — October 4, 2026

Authors and student numbers: Lars Heijnen [2233878]; Amirhossein Rezaei Bezenjani [2294753].

Report scope: Following CRISP-DM, we classify five-second windows of smartphone accelerometer, gyroscope and gravity data from 41 rides by two cyclists as smooth or bumpy lane surface, compare supervised and unsupervised methods, and apply the selected model to one unlabelled external recording.

Abstract

Uneven bicycle lanes make cycling less comfortable and can cause falls. We test whether the accelerometer, gyroscope and gravity sensor of a phone in a trouser pocket can tell smooth lanes from bumpy ones. Two cyclists recorded 41 labelled rides (22 bumpy, 19 smooth) in Eindhoven and Helmond. The complete recordings were aligned by timestamp and cut into 709 overlapping five-second windows with 121 statistical features. Ride-grouped cross-validation selected logistic regression with 86 features (macro F1 0.955). On eight held-out rides it reached 0.949 accuracy and 0.943 macro F1 over 136 windows, and a majority vote matched all eight ride labels. Four clustering methods mainly separated boundary motion and participant and device patterns, with weak agreement with road labels. The selected classifier predicted 385 bumpy and 21 smooth windows in the independent recording. Its gravity-axis features differ substantially from the training data, so these external predictions need labelled verification. Repeated locations and the two-rider sample also limit conclusions about transfer.

1 Business / Objective Understanding

Potholes, cracks and uneven surfaces make cycling uncomfortable and increase the risk of accidents and injuries, but inspecting every lane by hand is slow and expensive. Most cyclists carry a smartphone with motion sensors, so a model that recognises rough surfaces could turn ordinary rides into inspection data: a sequence of smooth or bumpy predictions that a maintenance team can map to decide where to inspect first. The task is close to human activity recognition for health monitoring, which also classifies short windows of body-worn inertial data and has to cope with differences between people, devices and phone placement.

We ask whether features from five-second sensor windows can separate smooth from bumpy lanes on rides the model has not seen. Following the assignment, smooth means a typical red cycling lane without imperfections, and bumpy means noticeable but safely passable imperfections such as small potholes or cracks (Eindhoven University of Technology, 2026, p. 3). Windows inherit the label of their ride, so short local changes in surface can be mislabelled, and placement, device, cyclist, speed and route all affect the signal. We therefore use ride-grouped validation with macro F1 and accuracy, analyse errors per window and per ride, and prefer a model that is interpretable and cheap enough to run on a phone. The external recording is used only after model selection.

2 Data Understanding

2.1 Collection protocol and dataset

P01 recorded with an iPhone 17 Pro and P02 with an iPhone 13 Pro, both in Sensor Logger (Logger Labs LTD, 2026) with a 10 ms interval (100 Hz) and standardisation enabled (Eindhoven University of Technology, 2026, p. 2). We use only the calibrated accelerometer, gyroscope and gravity streams. The phone was face down in a tight right trouser pocket to keep its placement and orientation as consistent as possible between rides. Riders kept a constant speed and the same gear where applicable. As the protocol prescribes, each recording starts before the phone goes into the pocket and ends after it is taken out again.

The rides were recorded in Eindhoven, including the TU/e campus, Vestdijk, Stratumsedijk and Kruisstraat, and three bumpy rides in Helmond. These names give the general location, meaning the road or cycle path, and rides at one location do not necessarily cover the same stretch. Several locations recur: P02 recorded ten smooth rides on the TU/e campus in one session and ten bumpy rides on Kruisstraat in another, and P01 recorded five smooth rides on Vestdijk. Rides from one location can share surface type and riding conditions, so the 41 recordings are not 41 independent locations. Each complete recording is labelled by the lane chosen for it. P01 contributed 12 bumpy and 7 smooth rides and P02 10 bumpy and 12 smooth, giving 22 bumpy and 19 smooth rides. Recordings last 32.3 to 56.1 s, 32.1 minutes in total (1,035 s bumpy, 893 s smooth). Every ride has a stable ID (R001 to R041). The processed files refer to the cyclists as P01 and P02, and the manifest keeps the source folder of each ride so it can be traced back to the raw recording.

2.2 Exploration and data quality

All 41 recordings (123 sensor files) share one schema: a nanosecond Unix timestamp, elapsed seconds, and x , y , z values in m/s^2 for acceleration and gravity or rad/s for the gyroscope. We did not use the uncalibrated streams that Sensor Logger also exported, and the annotation files were empty. No file contains missing or infinite values, duplicate rows or timestamps, or timestamps that fail to increase, so the missing-value treatments from the course (Kaymak, 2026a, slides 33–34) were not needed. Sampling rates lie between 99.58 and 99.94 Hz, and the largest gap between two samples is 13.8 ms, well under the 20 ms that one skipped sample would leave, which gives no evidence of a missing-sample gap. The only mismatch between sensors is R020, whose accelerometer ends one sample (10 ms) early, so streams are combined by timestamp.

Figure 2.1 summarises each recording with the magnitude $\sqrt{x^2 + y^2 + z^2}$, which does not depend on a single device axis. Within each participant, bumpy rides have more variable acceleration. We summarise every ride first and then take the median over rides. For P02, the median standard deviation of acceleration magnitude is 2.128 m/s^2 on bumpy rides against 1.581 on smooth rides, and the median of the ride means is 3.015 against 1.925 m/s^2 . The gyroscope overlaps far more, with a median standard deviation of 0.927 rad/s on bumpy and 0.978 on smooth P02 rides. Peaks separate the classes poorly: the largest, 40.72 m/s^2 , occurs in smooth ride R021 0.58 s after the start, like several other large smooth peaks. Such peaks can come from handling as well as from the road, so we kept them. Gravity magnitude stays near 9.80665 m/s^2 throughout.

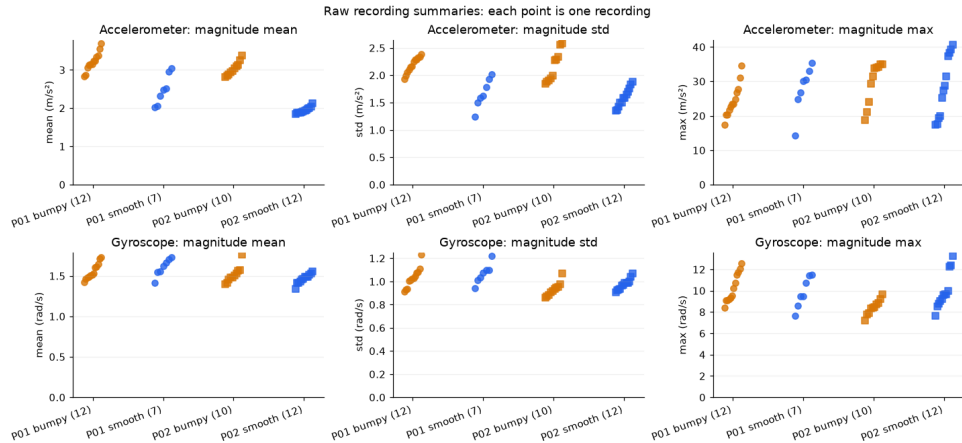


Figure 2.1: Magnitude summaries per recording by participant and class (orange bumpy, blue smooth).

Each file starts and ends with the phone being handled. Compared with a five-second segment in the middle, the first and last five seconds move clearly more (Figure 2.2). The median standard deviation of acceleration magnitude is 2.902 , 1.489 and 2.209 m/s^2 at the start, middle and end, and for the gyroscope 1.501 , 0.704 and 1.683 rad/s . R021, R014 and R019 have the largest start-to-middle ratios. The pattern occurs in both classes, which points to handling, but without start and stop labels we found no reproducible rule that separates handling from riding.

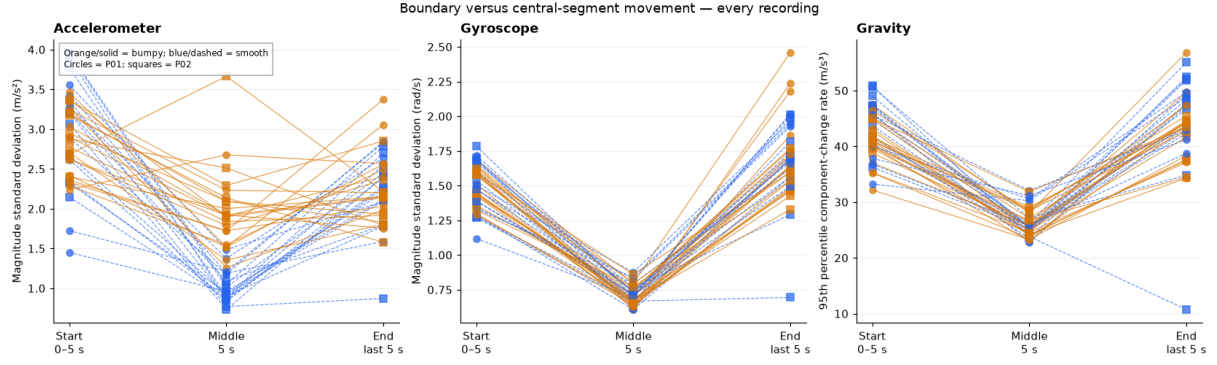


Figure 2.2: Movement in the first, middle and last five seconds of each ride (one line per ride; orange solid lines are bumpy rides, blue dashed lines smooth rides, circles P01 and squares P02). The gravity panel uses the 95th percentile of the component change rate.

The classes are roughly balanced (22 against 19 rides, 382 against 327 windows), so we did not rebalance them. Most limitations come from the collection. Device is tied to participant, ride labels miss short local changes, and the edges can contain handling. Location and recording day are also partly confounded with class: all of P02’s smooth rides were recorded on 8 September and all of its bumpy rides in one Kruisstraat session on 17 September, so part of P02’s class difference may come from the day rather than the road. P01 recorded 16 of its 19 rides on the morning of 8 September and gives the cleaner comparison. With two riders and two phones, we cannot tell whether the results hold for other people or devices.

3 Data Preparation

3.1 Preprocessing and windowing

The checks found no defects, so the export keeps every sample: we did not filter, clip, resample, scale, impute or crop. Smoothing (Kaymak, 2026a, slide 30) would damp the short vibrations that make a lane bumpy, clipping could remove real road events, and cropping by eye is not reproducible, so edge windows stay in as a possible source of label noise. A fixed boundary-window exclusion is tested separately on the training rides. The export (192,366 accelerometer and 192,367 gyroscope and gravity samples) has one CSV per ride and sensor and a manifest with ride, participant, label, device and unit. Quality reports and a read-back of every file confirm that values and timestamps are unchanged.

For feature extraction, the three streams of a ride are joined one-to-one on their timestamp (Kaymak, 2026a, slide 32), which only drops the unmatched last sample of R020 (192,366 aligned samples), and acceleration and gyroscope magnitudes are added. Windows of 500 samples (about 5 s) start every 250 samples (50% overlap), and partial windows are dropped. This yields 709 windows (382 bumpy, 327 smooth), 11 to 21 per ride, each with its ride’s label. Five seconds averages over pedalling and several irregularities while keeping a usable resolution along a route. The code checks increasing timestamps and a median rate between 95 and 105 Hz for every ride, so the exact window duration varies slightly by phone.

Overlapping windows of one ride are strongly dependent, so we split by ride. The eight test IDs are fixed as R002, R003, R020, R027, R028, R032, R038 and R040, preserving the original held-out rides across package versions. They contain 136 windows (89 bumpy, 47 smooth; four rides from each participant), against 33 training rides with 573 windows. Model selection uses four grouped folds within the training rides, so the test rides are used once.

3.2 Feature engineering and selection

For every window, 11 statistical features were calculated from the x , y and z axes of the three sensors and from the acceleration and gyroscope magnitudes. Gravity magnitude was left out because its norm stays almost constant near 9.80665 m/s^2 in every ride (Section 2.2). The features were mean, standard deviation, minimum, maximum, range, median, interquartile range, root mean square, mean absolute value, mean square and number of mean crossings. These windowed characteristics describe the level, variation, extreme values and oscillation of the sensor signals over time (Kaymak, 2026b, slides 29–30). In total, this gave 121 numerical features per window (11 signals with 11 statistics each).

Recording ID, participant ID, the road label and window time were kept only for tracing the data and creating the train/test split. They were not used as inputs for the classifiers.

Each cross-validation fold fits feature selection on its training rides only. The variance filter removes near-constant features; none were removed in the final fit. Remaining features are considered in descending training mutual information. A candidate is discarded if its absolute Pearson correlation with any previously retained feature exceeds 0.95. Standard scaling follows, then a second mutual-information selector keeps 20, 40 or all remaining features before classification. This count is tuned with the model settings. Variance, correlation and mutual information follow the feature-engineering lecture (Kaymak, 2026b, slides 8–18). In the final logistic-regression fit on 33 training rides, correlation filtering reduced 121 features to 86, and the selector kept all 86 (Figure 3.1).

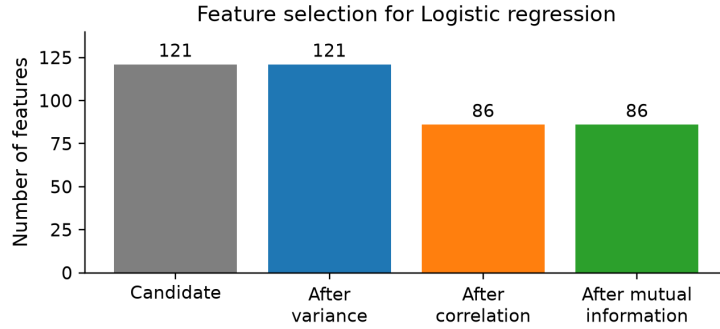


Figure 3.1: Number of features left after each selection step in the final logistic-regression fit.

4 Modeling

4.1 Supervised learning

We implemented four supervised classifiers from the course: logistic regression, k-nearest neighbours, a decision tree and a random forest (van der Sommen, 2026). Logistic regression is a linear model that combines weighted features into a class probability for each window. KNN predicts the class of a new window using nearby training windows. The decision tree learns threshold-based decisions from the features, while the random forest combines predictions from many decision trees.

We tuned each model with a small grid search: the regularisation strength $C \in \{0.1, 1, 10\}$ for logistic regression; $k \in \{5, 9, 15\}$ neighbours with uniform or distance weights for KNN; a maximum depth of 5 or 8 and at least 5 or 10 windows per leaf for the decision tree; and 200 trees with $\sqrt{p}$ or $0.5p$ candidate features per split and at least 3 or 5 windows per leaf for the random forest. The same search tuned the number of selected features. Standard scaling was fitted on the training folds only, which matters most for logistic regression and KNN because they depend on feature scale.

We tuned and compared the four models with the four ride-grouped folds on the 33 training rides from the data-preparation section, scored by macro F1.

Table 1: Grouped cross-validation performance: mean $\pm$ standard deviation across four tuning folds.

Model	Cross-validation accuracy	Cross-validation macro F1
Logistic regression	0.955 $\pm$ 0.007	0.955 $\pm$ 0.007
KNN, $k = 9$	0.918 $\pm$ 0.044	0.917 $\pm$ 0.044
Decision tree	0.947 $\pm$ 0.026	0.947 $\pm$ 0.026
Random forest	0.953 $\pm$ 0.026	0.953 $\pm$ 0.027

Logistic regression with $C = 10$ and all 86 retained features had the highest mean CV macro F1 (0.955), followed closely by the random forest (0.953). The gap of 0.0017 provides little separation between these configurations. We follow the selection rule set in advance and select logistic regression for the held-out test. Its coefficients describe each standardized feature’s contribution to the bumpy log odds, although correlated features can redistribute those coefficients. Phone runtime is not measured. The four tuned pipelines, including their fitted feature selection and scaling, are saved by the notebook so that the external recording can be scored without retraining.

4.2 Unsupervised learning

We implement K-means, agglomerative clustering, DBSCAN and a Gaussian mixture model (GMM), following the clustering and mixture-model lectures (Kaymak, 2026c, 2026d). K-means groups windows around centres, Ward agglomerative clustering merges groups by their increase in within-cluster variance, and DBSCAN identifies dense groups and noise. GMM estimates a mixture of distributions and gives soft component memberships.

Preparation uses the 33 training rides only. Without road labels, the variance and correlation filters retain 83 features, which are standardized before PCA. Keeping 95% of variance gives 24 components. Two clusters are the main hypothesis for K-means, Ward and GMM because the application has two road categories, but we also inspect two to five clusters. K-means uses 20 initializations. GMM compares diagonal, tied and full covariance with ten initializations per setting and selects the lowest training BIC among converged fits. Full covariance is selected. BIC assumes independent samples, so overlapping windows make it a configuration heuristic here.

DBSCAN compares min_samples of 5, 10 and 20 and eps values based on training nearest-neighbour distance quantiles, including eps=3 as a reference. A candidate must cover at least half the windows and form at least two clusters. Eligible candidates are ranked by coverage multiplied by assigned-only silhouette. The selected setting is eps=9.80 and min_samples=20. Road labels do not enter this search. The rule favours separation and coverage, but cannot guarantee that the groups describe road quality.

4.3 Supervised versus unsupervised comparison and final model choice

The supervised models use road labels directly. Clustering can describe recurring motion when labels are scarce, but its groups can also reflect the cyclist, phone orientation or handling. To compare predictive performance, K-means and GMM map each fitted cluster to its majority road label using training labels. Their filters, scaling, PCA and cluster-to-road mapping are learned within each of the same four grouped folds; GMM covariance is chosen within the fold. Validation labels enter scoring only. Ward clustering and DBSCAN have no native prediction method for unseen windows, so we evaluate their training partitions separately.

The final choice uses mean training CV macro F1: logistic regression scores 0.955, GMM 0.633 and K-means 0.415. We therefore deploy logistic regression. These CV scores were used for selection and are not independent performance estimates. The held-out comparison follows in Section 5.2. Saved pipelines retain the fitted transformations. Batch prediction takes 0.0011 to 0.0015 ms per window for the three predictive pipelines on this computer, excluding sensor-feature extraction. This does not establish phone speed or battery use.

5 Evaluation

5.1 Supervised-learning evaluation

Accuracy and macro F1 were used to evaluate the supervised models. Accuracy gives the total proportion of correct predictions. Macro F1 gives equal importance to smooth and bumpy windows by averaging the F1 scores of both classes.

The selected logistic regression was refitted on all 33 training rides and evaluated once on the eight held-out rides. It reached an accuracy of 0.949 and a macro F1 score of 0.943 over 136 windows. The majority-class baseline reached 0.654 accuracy and 0.396 macro F1. A majority vote over the windows of each ride matched all eight ride labels (five bumpy and three smooth).

Table 2: Class-wise performance of the selected logistic regression on the held-out test set.

Class	Precision	Recall	F1
Bumpy	0.966	0.955	0.960
Smooth	0.917	0.936	0.926
Macro average	0.941	0.946	0.943

The model predicted four bumpy windows in R002 as smooth and three smooth windows in R027 as bumpy (Figure 5.1). Six of these seven errors intersect the first or last five seconds; one is an interior window. This is consistent with the boundary checks, although handling is not annotated. The retained set contains 29 acceleration, 33 gyroscope and 24 gravity features. The largest absolute standardized

logistic coefficient belongs to `gravity_x_median`, so orientation contributes to the classifier as well as road motion. Ride-group splits prevent overlapping windows crossing folds; shared locations and sessions limit these scores to new rides under familiar collection conditions.

As a training-only sensitivity check, removing boundary windows raises macro F1 from 0.955 to 0.989 over 441 interior windows. Holding P01’s training rides out gives macro F1 0.737, and holding P02’s out gives 0.935. Both checks use the configuration selected with all training rides, so they are post-selection diagnostics. They point to boundary annotation and data from more riders and devices as next steps. The primary pipeline and test set stay unchanged.

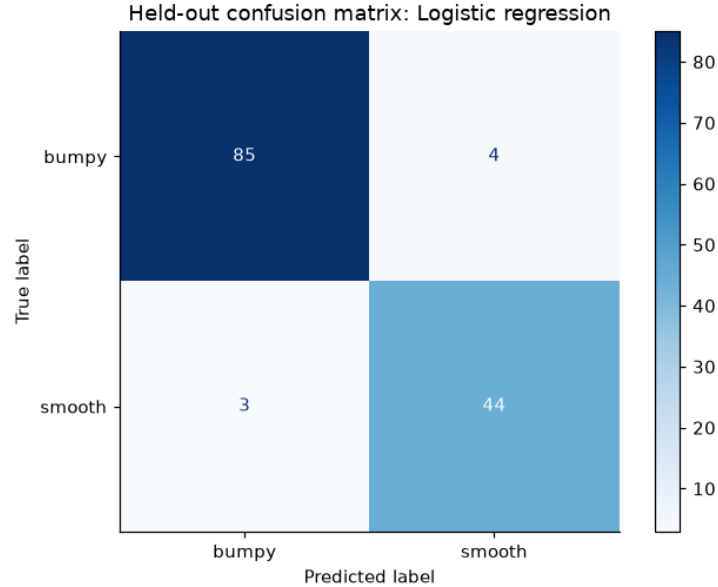


Figure 5.1: Confusion matrix of the selected logistic regression on the 136 held-out test windows.

5.2 Unsupervised-learning evaluation and final comparison

We evaluate the training partitions with purity, road-label adjusted Rand index (ARI), silhouette and coverage (Table 3). Purity matches each cluster to its most common road label and can favour small groups. ARI adjusts agreement for chance. Silhouette measures separation across all retained PCs, which need not follow the road labels. DBSCAN noise does not count as a correct purity match; all-window ARI treats it as one group, while silhouette uses assigned windows only.

Table 3: Training-cluster quality on 573 windows. Purity and road ARI include all windows; silhouette excludes DBSCAN noise.

Model	Purity	Road ARI	Silhouette	Coverage
K-means	0.511	-0.001	0.462	100%
Ward	0.511	-0.001	0.466	100%
DBSCAN	0.487	-0.001	0.495	95.1%
GMM	0.620	0.056	0.257	100%

K-means and Ward isolate groups of 64 and 62 boundary windows, respectively, with road ARI near zero. DBSCAN marks 28 windows as noise, all at boundaries and equally divided between road labels; its assigned-only purity is 0.512. GMM resembles participant, device and boundary structure more than road type (ARI 0.260 and 0.264 against road ARI 0.056). Full covariance uses 649 parameters in 24 dimensions, which is flexible relative to 33 training rides. These partitions help identify motion subgroups and candidate handling windows for annotation.

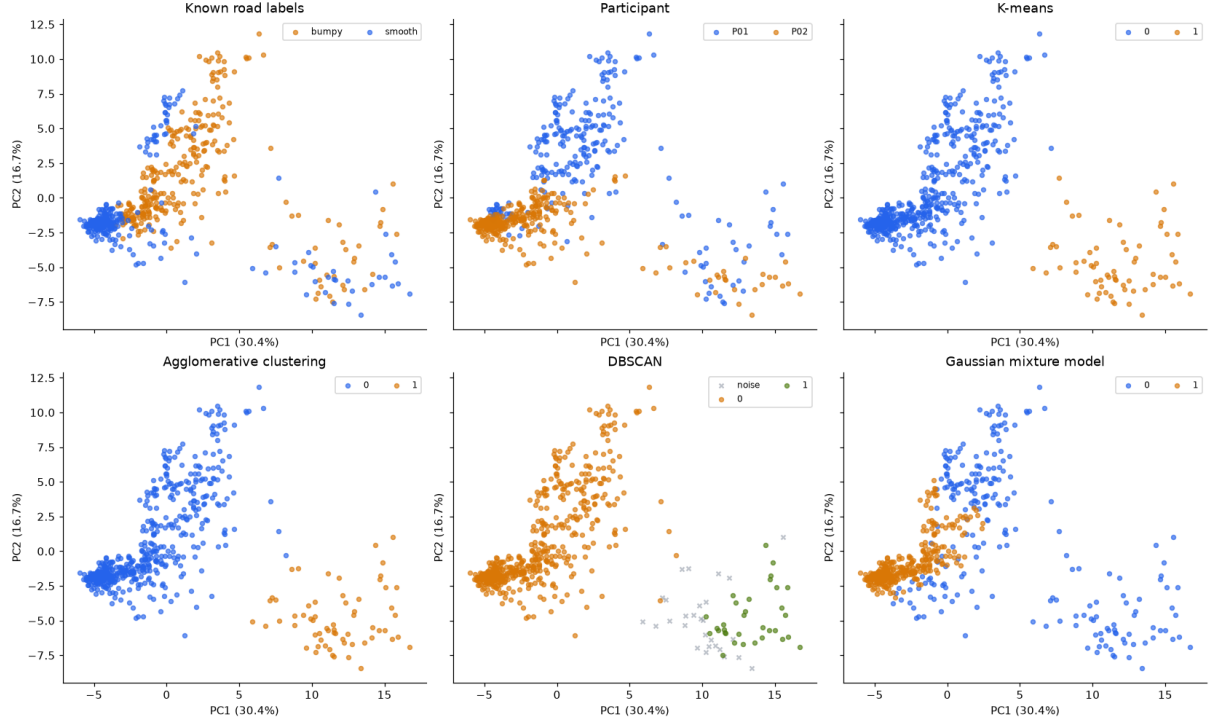


Figure 5.2: Road labels, participants and four clustering partitions in the same two-PC projection of training windows. The axes explain 47.1% of standardized variance; clustering uses 24 PCs. Grey crosses mark DBSCAN noise.

The five K-means seed checks give the same partition with 20 initializations, and retaining 90% variance gives the same K-means and Ward partitions in 15 PCs. Four K-means clusters raise road ARI to 0.424 while lowering silhouette to 0.254, suggesting additional motion subgroups. We keep the two-cluster comparison set in advance and do not select a count by road agreement.

Table 4: Predictive comparison on the same grouped folds and 136 held-out windows. Cluster-to-road mappings use training labels.

Model	CV macro F1	Test accuracy	Test macro F1
Logistic regression	0.955	0.949	0.943
GMM cluster-to-road	0.633	0.478	0.478
K-means cluster-to-road	0.415	0.654	0.396
Majority baseline	0.338	0.654	0.396

K-means maps both full-training clusters to bumpy and matches the baseline on the test rides. GMM has higher training purity but weak held-out prediction (Table 4). Majority ride voting is correct for 8/8 rides with logistic regression, 3/8 with GMM and 5/8 with K-means and the baseline. Labels make supervised learning suitable for the road-classification task, while clustering adds information about the representation and possible contamination. Longer rides contribute more windows, so the window metrics give them more weight.

6 Deployment Considerations and Recommendations

After model selection, the independent recording is checked and aligned with the same sensor schema and feature calculation. All 101,926 samples per sensor are retained by alignment, and the measured rate is 99.49 Hz. The frozen logistic pipeline predicts 385 bumpy and 21 smooth windows out of 406. No road labels are supplied, so deployment accuracy cannot be measured.

For a readable timeline, we average the bumpy scores of windows covering the midpoint of each disjoint one-second interval. A score of at least 0.5 gives a bumpy interval; uncovered intervals remain unknown. The longest predicted bumpy interval runs from 181.1 to 809.1 s (Figure 6.1). The averaging is retrospective because a covering window can include later samples. Five-second windows also limit the

resolution of local road events. Time alone cannot locate a road section without GPS or speed data.

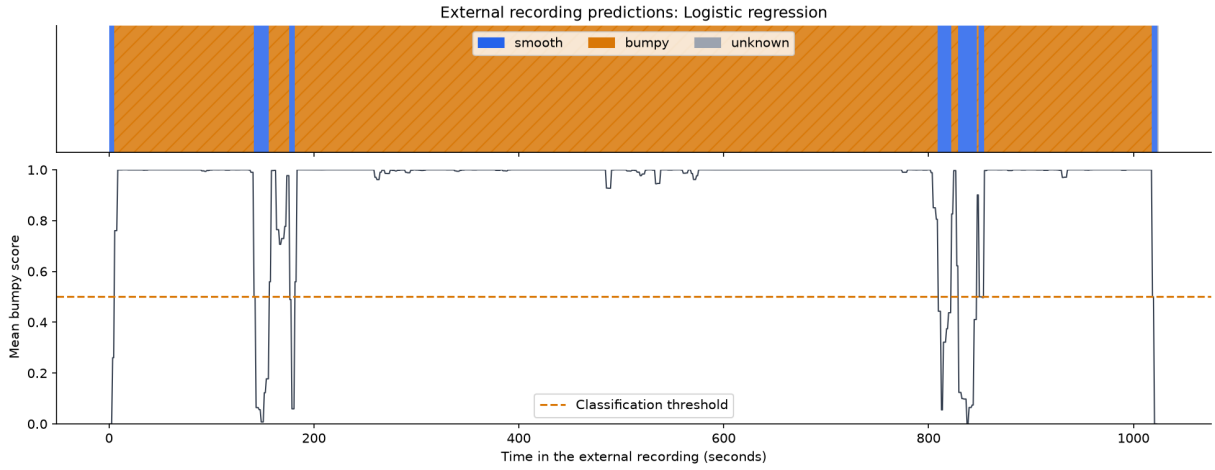


Figure 6.1: External recording predictions and mean bumpy scores from the selected logistic pipeline. Orange hatched intervals are predicted bumpy, blue smooth and grey unknown. These are unverified predictions.

Every external window's `gravity_y_mean` and `gravity_y_median` lies outside its training 1st to 99th percentile range. The external median `gravity_y_mean` is 3.211 m/s^2 , against a training range of -7.294 to -0.701 m/s^2 . Other gravity-axis summaries also shift. This is consistent with a different phone orientation, although placement is not measured independently. The mostly bumpy predictions may reflect unfamiliar inputs as well as road motion. The diagnostic is performed after selection and does not change the model or threshold.

A practical system would use these flags to choose sections for inspection after labelled external validation. Further collection should cover more riders, phones and routes, with both road classes repeated across days and annotated cycling boundaries. Route or session holdouts would test transfer more directly. An orientation-normalized or magnitude-focused feature comparison should be validated using new training data. A live prototype also needs causal updates, sensor-feature extraction timing and battery measurements; the current computer benchmark covers only batch prediction.

Technology statement

During the preparation of this assignment, Lars Heijnen and Amirhossein Rezaei Bezenjani used ChatGPT, mainly GPT-6 Sol and GPT-6.1 Sol, as a brainstorming partner for data preprocessing, exploratory data analysis, and feature engineering in the notebook. We also used it to refine and rewrite text and to check consistency between the report and notebook. We reviewed the outputs and edited the content as needed. We take full responsibility for the submitted work.

References

- Eindhoven University of Technology. (2026). *Assignment 1: Assessing bicycle lane quality using smartphone sensor data* [Assignment brief]. Canvas. <https://canvas.tue.nl/courses/37176>
- Kaymak, U. (2026a, September 4). *Basic data collection and pre-processing* [PowerPoint slides]. Canvas. <https://canvas.tue.nl/courses/37176>
- Kaymak, U. (2026b, September 8). *Feature engineering* [PowerPoint slides]. Canvas. <https://canvas.tue.nl/courses/37176>
- Kaymak, U. (2026c, September 11). *Unsupervised learning: Clustering* [PowerPoint slides]. Canvas. <https://canvas.tue.nl/courses/37176>
- Kaymak, U. (2026d, September 15). *Unsupervised learning: Mixture models and data visualization* [PowerPoint slides]. Canvas. <https://canvas.tue.nl/courses/37176>
- Logger Labs LTD. (2026). *Sensor Logger* (Version 1.65) [Mobile app]. App Store. <https://apps.apple.com/app/id1531582925>
- van der Sommen, F. (2026, September 18). *Supervised learning* [Lecture slides]. Canvas. <https://canvas.tue.nl/courses/37176>